

PROJECT ANALYSIS REPORT

DermatoAI – Skin Disease Prediction System using Deep Learning

Course: Artificial Neural Network & Deep Learning (ANNDL)
Final Term Assessment (TA) — Mini Project

Team Members

Roll No.	Name
TYAIDS-A41	Vedant Joshi
TYAIDS-A44	Rushikesh Kale
TYAIDS-A16	Aradhya Dike
TYAIDS-A11	Sanket Damale

GitHub: https://github.com/Aradhyadike/ANNDL/A16_TA2.pdf

1. Introduction

DermatoAI is a deep learning-powered web application designed to assist in the automated detection and prediction of skin diseases from dermatological images. Skin diseases are among the most prevalent medical conditions worldwide, yet accurate and timely diagnosis often requires specialist dermatologists who may not always be accessible to the general population. DermatoAI aims to bridge this gap by providing an intelligent, image-based diagnostic aid that can identify a range of skin conditions with high confidence.

The system leverages EfficientNet-B0, a state-of-the-art convolutional neural network architecture, trained on a dataset of approximately 27,000 labeled dermatological images spanning 10 distinct disease classes. These classes include common and serious conditions such as eczema, psoriasis, melanoma, and various fungal infections, among others. By training on a diverse and large-scale dataset, the model learns robust visual features that differentiate these conditions effectively.

The application is built as a full-stack web system. The backend is powered by FastAPI, a modern Python-based web framework known for its high performance and ease of use. The frontend is developed using React with Vite, offering a responsive and user-friendly interface. The two components communicate seamlessly via Axios-based HTTP calls, enabling real-time predictions from uploaded images.

This project demonstrates the practical application of deep learning in the medical imaging domain, and illustrates how modern AI techniques can be deployed as accessible web services. The report documents the complete technical pipeline including model architecture, training methodology, system design, and a critical evaluation of the project's strengths and limitations.

2. Objectives

The primary objectives of the DermatoAI project are as follows:

- Design and train a deep learning model capable of classifying skin disease images into 10 distinct diagnostic categories with high accuracy.
- Utilize the EfficientNet-B0 architecture, pretrained on large-scale image datasets, and fine-tune it on a domain-specific dermatological dataset of approximately 27,000 images.
- Develop a robust and scalable backend API using FastAPI that exposes a prediction endpoint capable of receiving image data and returning real-time inference results.
- Build an intuitive and responsive frontend interface using React and Vite, enabling users to upload skin images and receive diagnostic predictions without requiring medical expertise.
- Implement a confidence-based multi-prediction output system that returns the top 3 predicted disease classes along with their respective probability scores, aiding interpretability.
- Integrate the frontend and backend components using Axios for efficient and asynchronous HTTP communication.
- Demonstrate the feasibility and effectiveness of deploying a medical image classification model as a full-stack web application.

3. System Architecture & File Structure

DermatoAI follows a client-server architecture, with a clear separation between the AI inference backend and the user-facing frontend. The system is designed to be modular, scalable, and easy to maintain. Below is an overview of the major architectural components and their roles within the system:

Component	Technology	Key Responsibility
model.py / model file	PyTorch + EfficientNet-B0	Defines and loads the trained classification model
main.py (Backend)	FastAPI	Serves the /predict endpoint; handles image preprocessing and model inference
App.jsx (Frontend)	React + Vite	Provides the image upload interface and displays prediction results to the user
API Communication	Axios	Handles asynchronous HTTP POST requests from frontend to backend
Model Weights	.pth / checkpoint file	Stores the trained EfficientNet-B0 weights loaded at server startup

The overall data flow in the system is as follows: The user uploads a skin disease image through the React frontend. The frontend sends the image as a multipart form-data HTTP POST request to the FastAPI backend via Axios. The backend receives the image, applies the necessary preprocessing transformations (resize to 224×224, normalize), and passes it through the EfficientNet-B0 model. The model produces a softmax probability distribution over 10 classes, and the top 3 predictions with confidence scores are returned as a JSON response. The frontend then renders these results in a clean and readable format for the user.

4. Model Description

4.1 EfficientNet-B0 Architecture

The core of DermatoAI is built upon EfficientNet-B0, a highly efficient and accurate convolutional neural network architecture proposed by Tan and Le (2019). EfficientNet models are constructed using a compound scaling method that uniformly scales the network's depth, width, and resolution using a fixed set of scaling coefficients. This approach allows EfficientNet-B0 to achieve state-of-the-art accuracy while maintaining a significantly lower parameter count compared to other architectures such as ResNet or VGG.

EfficientNet-B0 is the baseline model in the EfficientNet family, consisting of a series of Mobile Inverted Bottleneck Convolution (MBConv) blocks. Each MBConv block incorporates depthwise separable convolutions and Squeeze-and-Excitation (SE) modules, which enable the network to learn channel-wise feature recalibration and capture fine-grained spatial patterns — a crucial capability for distinguishing visually similar skin conditions.

Parameter	Value	Details
Architecture	EfficientNet-B0	Compound-scaled CNN with MBConv blocks
Input Size	224 × 224 × 3	RGB image resized to standard input dimensions
Output Layer	Softmax (10 classes)	Produces probability distribution over 10 disease categories
Activation (hidden)	Swish / ReLU	Non-linear activations enabling complex feature learning
Pretrained Weights	ImageNet	Transfer learning baseline before fine-tuning on skin data

For the DermatoAI task, the final classification head of EfficientNet-B0 is replaced with a custom fully connected layer with 10 output neurons corresponding to the 10 skin disease categories. A Softmax activation function is applied at the output to produce a normalized probability distribution, enabling confidence-based multi-class prediction.

4.2 Output Classes

The model is trained to classify skin images into 10 distinct disease categories. These include conditions such as Eczema, Psoriasis, Melanoma, Fungal Infections, Acne, Rosacea, Seborrheic Dermatitis, Tinea Ringworm Candidiasis, Warts Molluscum Viral Infections, and other common dermatological conditions. The diversity of these classes necessitates a model with strong feature extraction capabilities, which EfficientNet-B0 provides through its deep MBConv block structure and SE attention mechanism.

5. Backend

5.1 FastAPI Server

The backend of DermatoAI is developed using FastAPI, one of the most performant Python web frameworks available for building REST APIs. FastAPI leverages Python's type annotation system to automatically generate API documentation (via Swagger UI and ReDoc) and provides asynchronous request handling, making it particularly well-suited for I/O-bound tasks such as serving deep learning model predictions.

The FastAPI server is responsible for receiving image upload requests from the frontend, performing the necessary preprocessing steps, running inference using the loaded EfficientNet-B0 model, and

returning structured prediction results. The server is designed to load the trained model weights once at startup, eliminating the overhead of reloading the model on every request and significantly reducing inference latency.

5.2 /predict Endpoint

The core API endpoint exposed by the backend is the /predict route, which accepts HTTP POST requests with a multipart form-data payload containing the uploaded skin image. Upon receiving the image, the backend performs the following operations sequentially:

- **Image Decoding:** The uploaded file is read and decoded from its byte stream into a PIL (Python Imaging Library) Image object.
- **Preprocessing:** The image is resized to 224×224 pixels to match the model's expected input dimensions. Standard normalization is applied using ImageNet mean and standard deviation values (mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]).
- **Tensor Conversion:** The preprocessed image is converted to a PyTorch tensor and an additional batch dimension is added to form a 4D tensor of shape [1, 3, 224, 224].
- **Inference:** The tensor is passed through the loaded EfficientNet-B0 model in evaluation mode (model.eval()) with gradient computation disabled (torch.no_grad()) for efficiency.
- **Softmax Output:** The raw logits from the output layer are passed through a Softmax function to obtain class probabilities.
- **Top-3 Extraction:** The top 3 predicted classes along with their confidence scores are extracted using PyTorch's topk() function and returned in a structured JSON response.

The JSON response includes an ordered list of the top 3 predicted disease classes, each accompanied by its predicted probability expressed as a percentage. This design allows the frontend to display multiple plausible diagnoses, improving the system's clinical utility.

5.3 Model Loading at Startup

To ensure low-latency responses, the trained model weights are loaded into memory when the FastAPI application starts, using FastAPI's startup event handler. The model is loaded onto the appropriate device (CPU or GPU) and set to evaluation mode. This approach ensures that subsequent prediction requests are handled without any model initialization delay, making the system responsive even under repeated usage.

6. Frontend

6.1 React + Vite Application

The frontend of DermatoAI is developed as a React single-page application (SPA) using the Vite build tool. Vite offers significantly faster development build times compared to traditional bundlers like Webpack, owing to its use of native ES modules during development. The React framework enables the development of a component-based, reactive user interface that updates dynamically in response to user interactions and API responses without requiring full page reloads.

The frontend is designed with clarity and ease of use as primary goals. Users with no technical background should be able to interact with the application intuitively. The interface guides users through the process of uploading an image and interpreting the displayed predictions.

6.2 Image Upload and Interaction

The frontend provides an image upload interface that allows users to select a dermatological image from their local file system. Upon selecting the image, a preview is rendered on the screen to provide immediate visual feedback. A 'Predict' or 'Analyze' button triggers the API call to the backend.

The uploaded image is packaged as a FormData object and transmitted to the FastAPI /predict endpoint via an Axios POST request. Axios is used for its simplicity, promise-based API, and automatic content-type handling for multipart form data. While the request is in progress, a loading indicator is displayed to inform the user that the analysis is underway.

6.3 Displaying Results

Once the prediction response is received from the backend, the frontend renders the top 3 predicted skin disease classes along with their associated confidence percentages. The results are displayed in a structured and visually clear format, making it easy for users to understand the most likely diagnosis and alternative possibilities. The clean UI design ensures that the prediction output is accessible and easy to read, even for non-technical users.

7. Training

7.1 Dataset

The EfficientNet-B0 model was trained on a curated dataset of approximately 27,000 dermatological images distributed across 10 skin disease categories. The dataset encompasses a wide variety of visual presentations for each condition, including variations in skin tone, lighting conditions, and disease severity. Such diversity in the training data is crucial for building a model that generalizes well to real-world images captured under varying conditions.

Standard data augmentation techniques were applied during training to further increase the effective diversity of the dataset and reduce the risk of overfitting. Common augmentations include random horizontal and vertical flips, random rotations, color jitter, and random cropping. These transformations help the model learn invariant features that are robust to superficial changes in the input images.

7.2 Training Configuration

Hyperparameter / Setting	Value / Description
Base Architecture	EfficientNet-B0 (pretrained on ImageNet)
Training Dataset Size	~27,000 images across 10 classes
Input Image Size	224 × 224 pixels (RGB)
Number of Epochs	2
Optimizer	Adam (Adaptive Moment Estimation)
Loss Function	CrossEntropyLoss
Output Activation	Softmax (10-class probability distribution)
Transfer Learning	Yes — pretrained ImageNet weights, fine-tuned on skin data

7.3 Optimizer and Loss Function

The Adam optimizer is employed for training, which combines the advantages of two gradient descent extensions: AdaGrad (which works well with sparse gradients) and RMSProp (which works well in online and non-stationary settings). Adam computes adaptive learning rates for each parameter and is widely regarded as a robust default optimizer for training deep neural networks on image classification tasks.

CrossEntropyLoss is used as the training objective. This loss function combines log-softmax and negative log-likelihood loss, making it well-suited for multi-class classification problems. It penalizes confident incorrect predictions more heavily, thereby encouraging the model to produce well-calibrated probability estimates across all 10 disease classes.

7.4 Transfer Learning Strategy

Rather than training EfficientNet-B0 from scratch, the project leverages transfer learning by initializing the network with weights pretrained on the ImageNet dataset. ImageNet-pretrained models have already learned rich, generalizable feature representations from millions of diverse images. By fine-tuning these weights on the dermatological dataset, the model benefits from low-level feature detectors (edge detectors, texture filters) learned from ImageNet, while adapting higher-level representations to skin disease-specific visual patterns. This strategy is especially beneficial when the target dataset (27,000 images) is smaller than what would typically be required to train a deep CNN from random initialization.

8. Technology Stack

Component	Technology / Library
Programming Language	Python 3.x
Deep Learning Framework	PyTorch (torch, torchvision, torch.nn, torch.optim)
Model Architecture	EfficientNet-B0 (via torchvision.models)
Backend Framework	FastAPI
Frontend Framework	React (with Vite build tool)
API Communication	Axios (HTTP client for JavaScript)
Image Processing	PIL / Pillow, torchvision.transforms
Training Optimizer	Adam (torch.optim.Adam)
Loss Function	CrossEntropyLoss (torch.nn.CrossEntropyLoss)
Numerical Computing	NumPy

9. Strengths of the Project

- End-to-End System Implementation:** DermatoAI covers the complete machine learning deployment pipeline — from dataset-based model training and fine-tuning, through backend API development, to a fully functional frontend interface. This demonstrates a comprehensive understanding of both AI and software engineering principles.
- State-of-the-Art Model Architecture:** The use of EfficientNet-B0 brings significant advantages in terms of accuracy-to-parameter ratio. Its compound scaling and MBConv block design make it one of the most effective lightweight CNNs available for image classification tasks.
- Transfer Learning Utilization:** By leveraging ImageNet-pretrained weights, the project significantly reduces the amount of training data required to achieve acceptable accuracy, and accelerates convergence. This is a best practice in medical imaging tasks where large labeled datasets are difficult to obtain.

- **Real-Time Web Deployment:** The integration of FastAPI and React with Vite results in a low-latency, responsive web application. Users receive prediction results in near real-time after uploading an image, making the system practical for interactive use.
- **Multi-Class Top-3 Predictions with Confidence Scores:** Providing the top 3 predictions with associated confidence percentages rather than a single output greatly improves the clinical interpretability of the system. This allows users to consider alternative diagnoses, which is particularly valuable in a medical context.
- **Clean and User-Friendly Interface:** The React frontend is designed with user experience in mind, offering a clean layout that guides users through the process without requiring technical expertise. The image preview and results display are both intuitive and informative.
- **Modular and Maintainable Architecture:** The clear separation of concerns between the model, backend API, and frontend application ensures that each component can be updated or scaled independently. This modularity is a significant advantage for future development and maintenance.

10. Limitations & Scope for Improvement

- **Limited Training Epochs:** The model was trained for only 2 epochs, which is significantly fewer than what is typically required to fully fine-tune a pretrained CNN on a new domain. More training epochs with a suitable learning rate schedule (such as cosine annealing or ReduceLROnPlateau) could substantially improve the model's accuracy and generalization.
- **Absence of Validation and Test Set Reporting:** The report does not include quantitative evaluation metrics such as accuracy, precision, recall, F1-score, or confusion matrix on a held-out validation or test set. Without these metrics, it is difficult to objectively assess the model's real-world performance and identify class-wise weaknesses.
- **Dataset Class Imbalance:** Dermatological datasets often suffer from significant class imbalance, with some conditions being far more represented than others. Without explicit handling through class-weighted loss or oversampling techniques (e.g., SMOTE), the model may be biased towards more frequent classes and perform poorly on rarer conditions.
- **No Model Explainability:** The current implementation does not include any explainability or interpretability mechanisms such as Grad-CAM or LIME. These techniques generate saliency maps that highlight the image regions the model focuses on when making a prediction, which is crucial for building trust in medical AI applications.
- **No Input Validation or Error Handling:** The backend does not appear to include robust input validation to handle non-image uploads, corrupted files, or images of unsupported formats. Adding comprehensive error handling would make the system more resilient and production-ready.
- **Limited Training Duration May Affect Accuracy:** With only 2 epochs of training on ~27,000 images, the model may not have converged to its optimal weights. Extended training with early stopping based on validation loss would likely yield a more accurate and reliable model.
- **No User Authentication or Data Privacy Measures:** Since the application processes medical images, it should ideally incorporate user authentication and data privacy safeguards to comply with healthcare data regulations. The current implementation does not address these concerns, limiting its deployment in clinical environments.
- **Lack of Mobile Responsiveness:** While the React frontend provides a clean desktop interface, the application has not been explicitly optimized for mobile devices. Since many users may attempt to upload photos taken directly from smartphones, responsive design and mobile optimization would be a valuable enhancement.

11. Conclusion

DermatoAI successfully demonstrates the application of deep learning to the domain of medical image analysis, specifically for skin disease classification. The project brings together a well-established CNN architecture (EfficientNet-B0), a scalable backend API (FastAPI), and a modern frontend framework (React with Vite) to deliver a complete, functioning web application that can provide real-time diagnostic predictions from dermatological images.

The use of transfer learning on a dataset of approximately 27,000 images across 10 disease classes represents a practical and effective approach for building medical image classifiers in resource-constrained academic settings. The top-3 multi-class prediction output with confidence scores adds meaningful interpretability to the system, which is a critical consideration in healthcare applications.

While the current implementation has notable limitations — particularly regarding training depth, evaluation rigor, and clinical safeguards — the project establishes a strong and extensible technical foundation. With enhancements such as extended training with learning rate scheduling, Grad-CAM explainability integration, class imbalance handling, and comprehensive model evaluation, DermatoAI has the potential to evolve into a significantly more accurate and clinically valuable diagnostic aid.

Overall, DermatoAI is a well-conceived and well-executed mini-project that effectively integrates ANNDL concepts with modern full-stack web development practices. It reflects a clear understanding of the end-to-end machine learning deployment pipeline and represents a meaningful contribution to the exploration of AI-assisted dermatological diagnosis.

End of Report